

ClearPath System Architecture

Document: 02 of 10
Phase: MVP
Status: Complete
Generated: 2026-04-04

Overview

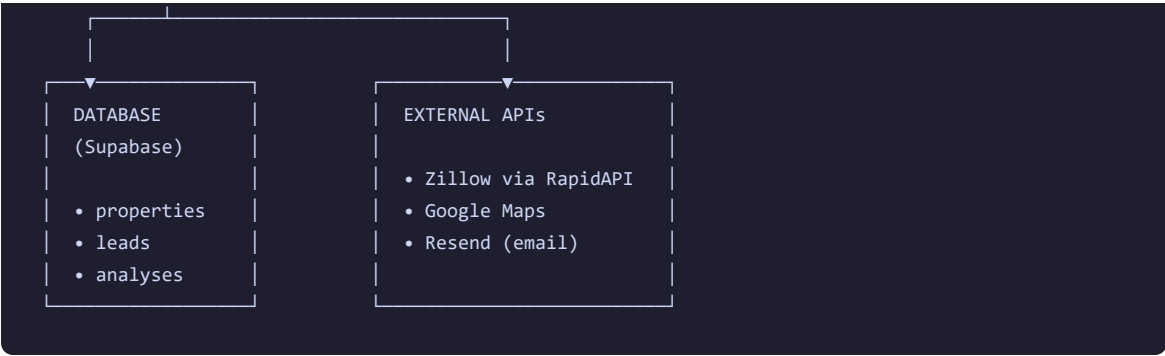
ClearPath is a two-system platform built on a shared infrastructure but kept logically separate:

SYSTEM	SCOPE	PURPOSE
ClearPath Analyzer	Global	Free tool — attract and qualify investors
ClearPath Asset Group	Chicago only	Paid service — execute deals for qualified investors

Architecture principle: Simple beats clever. Every layer is chosen for speed to ship, not engineering prestige.

System Diagram





Tech Stack

Frontend

LAYER	CHOICE	WHY
Framework	Next.js 14 (App Router)	SSR for SEO, fast load, React ecosystem
Styling	Tailwind CSS	Utility-first, fast to build, mobile-first
Forms	React Hook Form	Lightweight, easy validation
State	React useState/useContext	No Redux needed for MVP
Hosting	Vercel	Zero-config Next.js deployment

Backend

LAYER	CHOICE	WHY
API	Next.js API Routes	No separate server needed, collocated
Calculation Engine	Pure JavaScript	No dependencies, fast, testable
Auth (Phase 3)	Supabase Auth	Free tier, simple

Database

LAYER	CHOICE	WHY
DB	Supabase (PostgreSQL)	Free tier, instant REST API, easy setup
ORM	Supabase JS Client	Direct, no extra layer

External Services

SERVICE	PROVIDER	PURPOSE
Property Data	RapidAPI → Zillow API	Pull address data, Zestimate, sq ft, beds/baths
Address Validation	Google Maps Places API	Autocomplete + lat/long
Email	Resend	Transactional email (lead capture confirmation)
Analytics	Plausible (or Vercel Analytics)	Simple, privacy-friendly

Application Pages

ClearPath Analyzer

```
/ Landing Page
|
├─ Hero: "Analyze Any Deal in 60 Seconds"
├─ Analyzer Form (address or Zillow link)
├─ How It Works (3 steps)
├─ Social proof
└─ Footer

/analyze Deal Input Page (if standalone form)
|
├─ Address input (Google autocomplete)
├─ Condition selector (cosmetic / light / medium / heavy / gut)
├─ Purchase price input
├─ Optional: down payment %, hold months
└─ Submit → triggers /api/analyze

/results Results Page
|
├─ Property summary (address, beds, baths, sqft)
├─ ARV card
├─ Rehab Estimate card (range)
├─ Rent Estimate card
├─ Cash Flow card (+ deal signal)
├─ Flip Profit card (+ MAO, deal signal)
├─ Email capture gate (show full results after email)
└─ If Chicago address → Asset Group CTA banner
```

ClearPath Asset Group

```
/asset-group Service Landing Page
|
├─ Hero: "We Execute Chicago Deals For You"
├─ Service breakdown (Rehab / Lease-Up / Stabilization)
├─ Who it's for
└─ How it works (3 steps)
```

- └─ CTA: "Book a Call" → Calendly embed or form
- └─ Footer

Backend: API Routes

POST /api/analyze

Purpose: Orchestrates the full deal calculation.

Request:

```
{
  "address": "1234 W Chicago Ave, Chicago, IL 60622",
  "condition": "medium",
  "purchase_price": 120000,
  "down_payment_pct": 0.25,
  "hold_months": 6
}
```

Process:

1. Validate inputs
2. Call /api/property to fetch property data
3. Run ARV calculation
4. Run Rehab Estimate
5. Run Rent Estimate
6. Run Cash Flow
7. Run Flip Profit
8. Generate deal signals
9. Save analysis to DB (async, non-blocking)
10. Return result object

Response:

```
{
  "property": {
    "address": "1234 W Chicago Ave",
    "sqft": 1400,
    "bedrooms": 3,
    "bathrooms": 2,
    "year_built": 1965,
    "is_chicago": true
  },
  "results": {
    "arv": 185000,
    "rehab_low": 28000,
    "rehab_high": 44000,
  }
}
```

```

    "rent_estimate": 1665,
    "monthly_cash_flow": 247,
    "cash_on_cash_return": 6.2,
    "flip_profit": 34200,
    "mao": 154500,
    "deal_signal_rental": "green",
    "deal_signal_flip": "green"
  }
}

```

GET /api/property

Purpose: Fetch property data from Zillow via RapidAPI.

Process:

1. Accept address string
2. Call RapidAPI Zillow endpoint
3. Extract: sqft, beds, baths, year_built, zestimate, lat/lng
4. Detect if Chicago (check city field or zip code range)
5. Return normalized property object

Fallback: If API fails or property not found, return empty object → frontend prompts user to enter sq ft manually.

POST /api/leads

Purpose: Save a lead after email capture.

Request:

```

{
  "email": "investor@email.com",
  "name": "John Smith",
  "address": "1234 W Chicago Ave",
  "analysis_id": "uuid",
  "is_chicago": true
}

```

Process:

1. Validate email
2. Insert into leads table
3. Send confirmation email via Resend
4. If is_chicago → tag lead as "chicago_prospect"
5. Return success

Database Schema (Preview — full schema in Document 09)

analyses table

id	UUID	PRIMARY KEY
address	TEXT	
property_data	JSONB	
inputs	JSONB	-- condition, purchase_price, etc.
results	JSONB	-- all calculated outputs
created_at	TIMESTAMP	

leads table

id	UUID	PRIMARY KEY
email	TEXT	UNIQUE
name	TEXT	
analysis_id	UUID	REFERENCES analyses(id)
is_chicago	BOOLEAN	
source	TEXT	-- 'analyzer', 'asset_group', 'content'
status	TEXT	-- 'new', 'contacted', 'qualified', 'client'
created_at	TIMESTAMP	

Data Flow

Step 1: User enters address on landing page
Step 2: Google Maps autocomplete validates and formats address
Step 3: User submits form → POST /api/analyze fires
Step 4: Backend fetches property data from Zillow ↳ Fails → User prompted to enter sqft manually
Step 5: Calculation engine runs all 5 formulas
Step 6: Results stored in analyses table (async)
Step 7: Results returned to frontend ↳ Partial results shown immediately (ARV, rehab) ↳ Email gate shown before full results unlock
Step 8: User enters email → POST /api/leads fires

```
    ↳ Lead saved to DB
    ↳ Confirmation email sent

Step 9: Full results displayed
    ↳ If Chicago address → Asset Group CTA shown
```

Environment Variables

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# RapidAPI (Zillow)
RAPIDAPI_KEY=
RAPIDAPI_HOST=zillow-com1.p.rapidapi.com

# Google Maps
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY=

# Resend (Email)
RESEND_API_KEY=

# App
NEXT_PUBLIC_APP_URL=https://clearpath.com
```

Deployment

ENVIRONMENT	PLATFORM	BRANCH
Production	Vercel	main
Preview	Vercel	dev (auto-deploy on PR)
Local	npm run dev	—

Deploy steps:

```
1. Push to GitHub
2. Connect repo to Vercel
3. Add all environment variables in Vercel dashboard
4. Deploy
```

MVP Scope Boundary

✓ In MVP

- Analyzer form + results page
- All 5 deal calculations
- Email capture
- Lead saved to DB
- Confirmation email
- Asset Group landing page
- Chicago CTA trigger

✗ Out of MVP (Phase 2+)

- User accounts / login
- Saved analyses history
- Dashboard
- Comp pulling (use Zestimate fallback for MVP)
- SMS automation
- Advanced CRM integrations
- Admin dashboard

Security Considerations

RISK	MITIGATION
API key exposure	All keys server-side only (no NEXT_PUBLIC_ except maps/supabase anon)
Rate limit abuse	Rate limit /api/analyze to 10 req/IP/hour
Email spam	Email validation + Supabase RLS on leads table
SQL injection	Supabase parameterized queries by default

Next Document → [03_ui_wireframes.md](#) — UI / UX Wireframes